**COLLEGE DE PARIS SUPERIEUR**

**Bachelor**

*Tronc commun*

**RÉPUBLIQUE TOGOLAISE**

*Travail – Liberté – Patrie*

# INTRODUCTION À LA PROGRAMMATION

## en langage Python

Code : | Semestre 1 | 24h | 2 ECTS

**De l'algorithmique à l'écriture de scripts Python**

Variables, conditions, boucles, fonctions, structures de données, fichiers, exceptions

Python IDE Débogage

**Niveau :** Bachelor 1 (B1) – Licence en IT

**Dr. AFENYIVEH Dzo M. Serge**

*dzo.serge@gmail.com / d.afenyiveh@univkara.com*

*(+228) 91 75 98 33 / 98 35 23 31*

14 décembre 2025

*Année académique : 2025–2026*

# Sommaire

**SYLLABUS DU COURS** ..... **1**

**1 Module 1 : Introduction et environnement Python** ..... **6**
1.1 Historique, philosophie et caractéristiques de Python ..... 6
1.2 Installation et gestion des packages Python ..... 9
1.3 Découverte du REPL et premières commandes : Votre premier laboratoire Python ..... 14
1.4 Environnements de développement (IDEs) : Votre poste de travail Python complet ..... 17
1.5 Structure d'un programme Python : Organiser votre premier code. 22
1.6 Variables et types de données de base : Les fondations de l'information IT ..... 25
1.7 Travaux Pratiques (TP) du Module 1 : Mettre les mains dans le code 32
1.8 Quiz du Module 1 : Testez vos connaissances sur Python et son environnement ..... 35

# Syllabus du Cours

## Programmation Python

### Introduction à la Programmation en Langage Python

Bachelor 1 (B1) – Tronc commun

### Description du Cours

Ce cours initie les étudiants aux concepts fondamentaux de la programmation en utilisant **Python**, un langage polyvalent et accessible, largement utilisé en *science des données*, *développement web* et *automatisation*.

**Python** **Scripts** **Débogage** **Mini-projets**

## Informations Générales

| | |
|---|---|
| **Intitulé** | Introduction à la Programmation en Langage Python |
| **# Code** | B1COM0304 |
| **Niveau** | Bachelor 1 (B1) – Tronc commun |
| **Durée** | 24 heures (CM + TD + TP) |
| **Crédits** | 2 |
| **Semestre** | Semestre 1 |
| **Prérequis** | › Introduction à l'Algorithmique et structure de données |

## Objectifs Pédagogiques

À l'issue de ce cours, l'étudiant doit être capable de :

- ✅ Comprendre la syntaxe et les concepts fondamentaux du langage Python.
- ✅ Écrire et exécuter des scripts Python simples pour résoudre des problèmes.
- ✅ Utiliser les structures de données principales : listes, dictionnaires, tuples et ensembles.
- ✅ Appliquer les structures de contrôle pour la logique de programmation.
- ✅ Déboguer et tester des programmes Python.

## Contenu Pédagogique

### 1. Introduction au langage Python

- Historique et applications du langage Python
- Installation de Python et prise en main de l'environnement de développement (IDE)
- Structure d'un programme Python
- Types de données de base et variables

### 2. Structures de contrôle

- Les conditions : `if`, `else`, `elif`
- Les boucles : `for`, `while`
- Utilisation des boucles pour itérer sur des structures de données

### 3. Fonctions en Python

- Définition et appel de fonctions
- Paramètres et retour de valeurs
- Portée des variables (locale et globale)
- Fonctions intégrées et modules standards (`math`, `random`, etc.)

### 4. Structures de données

- Listes et opérations associées (ajout, suppression, tri, etc.)
- Dictionnaires : création, accès et manipulation
- Tuples et ensembles : caractéristiques et utilisation
- Applications pratiques des structures de données

### 5. Introduction à la manipulation des fichiers

- Lecture et écriture dans les fichiers
- Gestion des erreurs lors de l'accès aux fichiers
- Utilisation de fichiers pour stocker et récupérer des données

### 6. Introduction au débogage et bonnes pratiques

- Techniques de débogage de base
- Gestion des exceptions (`try`, `except`)
- Introduction à l'utilisation des bibliothèques
- Documentation et structuration du code
- Bonnes pratiques pour écrire un code Python clair et lisible

## Approches Pédagogiques

- Exercices de programmation à chaque séance pour pratiquer les concepts.
- Utilisation de notebooks interactifs (Jupyter/Colab) pour faciliter l'expérimentation.
- Travaux dirigés orientés vers la résolution de problèmes pratiques avec Python.
- Mini-projets pour renforcer les compétences en fin de cours.

## Méthode d'évaluation

- **Contrôle(s) continu(s) (TD ou TP)** : **60%** de la note finale
- **Examen final ou Projet Final** : **40%** de la note finale

## Références Bibliographiques


[1] **Zelle, J.** *Python Programming : An Introduction to Computer Science.* Franklin, Beedle & Associates, 2010.

[2] **Matthes, E.** *Python Crash Course.* No Starch Press, 2016.

[3] **Sweigart, A.** *Automate the Boring Stuff with Python.* No Starch Press, 2015.

[4] **Lutz, M.** *Learning Python.* O'Reilly Media, 2013.

> *« Programmer, c'est apprendre à décomposer un problème en étapes simples et vérifiables. »*

CHAPITRE I

# MODULE 1 : INTRODUCTION ET ENVIRONNEMENT PYTHON

**Objectifs d'apprentissage**

À l'issue de ce chapitre, vous serez capable de :

- ✓Connaître l'historique, la philosophie et les principes fondamentaux de Python.
- ✓Comprendre la place centrale de Python dans les métiers de l'IT : développement web, data science, automatisation, etc.
- ✓Maîtriser l'installation de Python et la gestion des bibliothèques (`pip`, `conda`).
- ✓Utiliser la console interactive (REPL) pour des tests rapides et exécuter des scripts Python.
- ✓Interagir avec l'utilisateur via les fonctions `print()` et `input()`.
- ✓Choisir et configurer un environnement de développement (IDE) adapté à vos projets IT.

## 1.1 Historique, philosophie et caractéristiques de Python

Python est un langage de programmation **de haut niveau**, **interprété**, **orienté objet** et **multiparadigme**. Son objectif central est la **lisibilité** : un code plus facile à relire se maintient mieux et coûte moins cher à faire évoluer, ce qui est déterminant en **Information Technology**. Le langage a été initié par **Guido van Rossum** en 1989 au *CWI* (Centrum Wiskunde & Informatica, Pays-Bas) et rendu public en 1991.

La philosophie *"batteries included"* signifie que Python fournit dès l'installation une bibliothèque standard riche (fichiers, JSON, réseau, dates, tests,

etc.). En IT, cela se traduit par un gain immédiat : moins de dépendances à ajouter pour produire un script utile.

**Le Zen de Python : principes utiles en ingénierie logicielle**

Le *Zen de Python* (Tim Peters) est accessible via `import this`. Il sert de guide pour écrire du code exploitable en équipe :

- **Beau est mieux que laid. Utilité IT :** La lisibilité réduit le temps de revue, de maintenance et le taux d'erreurs.
- **Explicite est mieux qu'implicite. Utilité IT :** Un comportement explicite diminue les bugs et facilite le débogage en production.
- **Simple est mieux que complexe. Utilité IT :** La complexité augmente les risques (pannes, vulnérabilités, coûts).
- **La lisibilité compte. Utilité IT :** L'onboarding et la collaboration (Agile/DevOps) deviennent plus rapides.

### 1.1.1 Sous le capot : interprété vs compilé (utile pour Dev/-Debug)

En informatique, la manière dont un langage est exécuté est fondamentale pour la performance, le déploiement et le débogage. **Python est un langage interprété**. Contrairement aux langages compilés (comme C++, Java où le code source est transformé en un fichier binaire exécutable avant d'être lancé), Python exécute le code via un programme intermédiaire : l'interpréteur. Il lit, analyse et exécute chaque instruction ligne par ligne, ou en convertit le code source en un "bytecode" intermédiaire exécuté par une machine virtuelle Python.

Cette distinction a des implications directes sur votre travail en IT :

- **Cycle de développement (DevOps) :** Rapidité d'essai et de modification du code. Pas de longue phase de compilation.
- **Déploiement et Distribution :** Moins de fichiers binaires à gérer, mais nécessite un interpréteur Python sur la machine cible.
- **Débogage (Debugging) :** Facilite l'identification des erreurs, car elles sont souvent détectées à l'exécution de la ligne concernée.
- **Performance :** Généralement plus lent à l'exécution qu'un langage compilé, mais la productivité du développeur est accrue. Pour pallier cela, les bibliothèques critiques de Python sont souvent écrites en C/C++ et "liées"

(bindées) à Python.

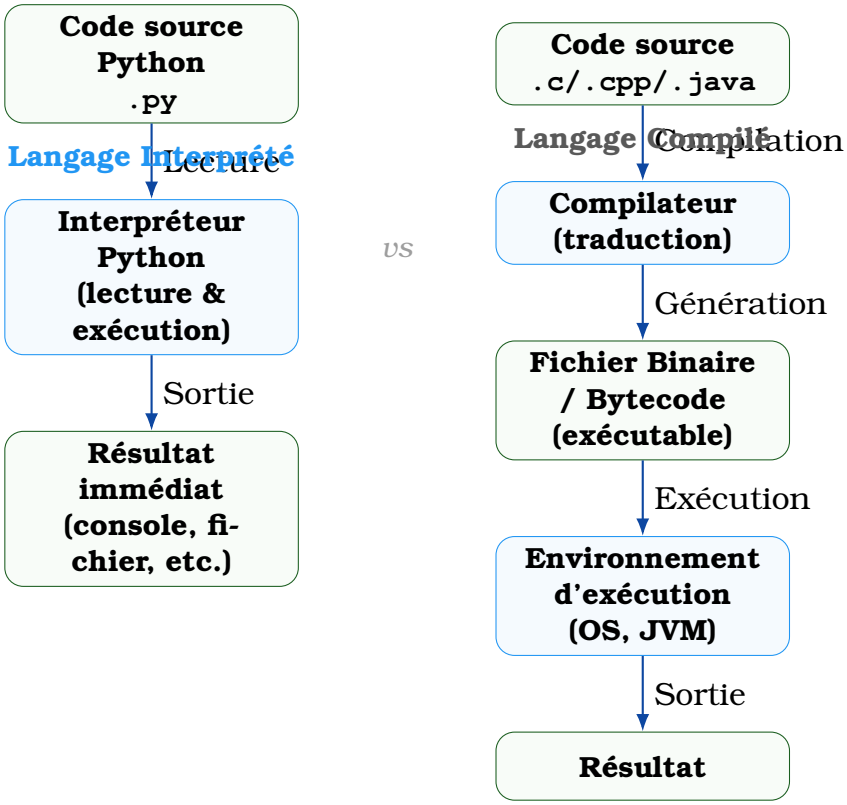



**FIGURE 1.1 –** *Comparaison des chaînes d'exécution : interprété (Python) vs compilé (ex : C++, Java).*

**Utilité IT immédiate**

Python est le choix privilégié pour les scripts d'administration système, les outils DevOps, les prototypes rapides d'applications web (APIs) ou les tests unitaires. Pour les tâches nécessitant des performances extrêmes (calculs intensifs, rendu graphique), on s'appuie sur des bibliothèques Python optimisées en C/C++ (comme NumPy) ou des architectures distribuées.

### 1.1.2 Carte conceptuelle : caractéristiques et bénéfices IT

Python offre un ensemble de caractéristiques qui se traduisent par des avantages directs et concrets pour le développeur et l'ingénieur IT. Comprendre cette synergie est essentiel pour exploiter pleinement le langage.

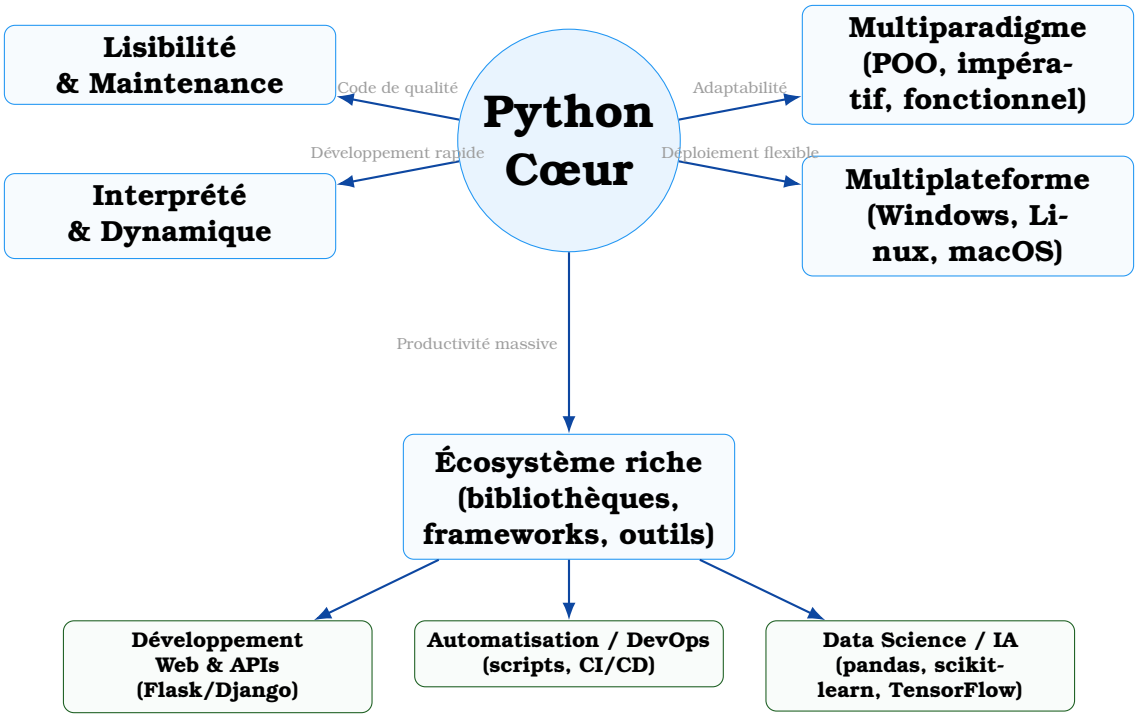



**FIGURE 1.2 –** *Carte conceptuelle des caractéristiques de Python et leurs bénéfices directs en IT.*

### 1.1.3 Chronologie : versions, maturité et rôle en IT

L'évolution de Python reflète sa capacité à s'adapter aux besoins croissants de l'industrie informatique. Comprendre cette chronologie permet d'apprécier sa robustesse actuelle.

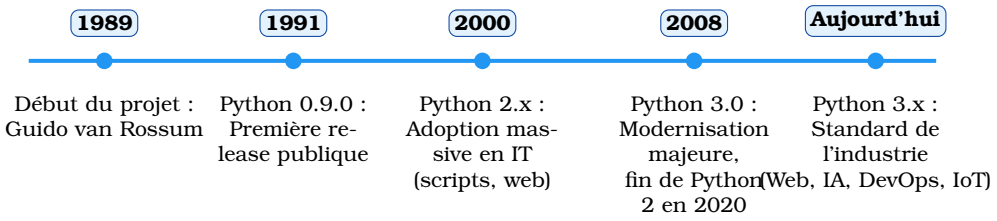



**FIGURE 1.3 –** *Chronologie simplifiée : Python, de son origine à son rôle de standard IT.*

## 1.2 Installation et gestion des packages Python

L'installation de Python et la gestion efficace de ses bibliothèques (aussi appelées "packages" ou "modules") sont des étapes fondamentales. Une configuration propre et bien organisée est la première pierre angulaire de tout projet IT réussi. Elle garantit la stabilité, la reproductibilité de vos développements et

évite les "conflits de dépendances" qui peuvent rapidement devenir un casse-tête.

### 1.2.1 Téléchargement et installation de Python

**Choix recommandé**

Pour un étudiant en **Bachelor IT**, deux voies sont réalistes :

- **Anaconda/Miniconda** : recommandé si vous visez **Data/ML** (dépendances compilées, environnements conda).
- **Python officiel** : idéal pour **scripts, web, automatisation** (léger, standard).

**Option A — Anaconda / Miniconda**

**Profil :** Data Science, ML, notebooks
**Inclus :** `conda`, Jupyter, libs scientifiques

**Points forts :**
- gestion d'environnements robuste
- dépendances natives mieux gérées
- démarrage rapide pour data/IA

**Téléchargement :** https://www.anaconda.com/download

**Option B — Python officiel**

**Profil :** IT général (scripts, web, automation)
**Inclus :** `pip` (PyPI), `venv`

**Points forts :**
- installation légère et standard
- excellent pour projets web/API
- workflow pro via `venv` + `pip`

**Téléchargement :** https://www.python.org/downloads/

**FIGURE 1.4 –** *Deux chemins d'installation : choisissez selon le type de projets IT visés.*

TABLE 1.1 – *Comparatif rapide : Anaconda/Miniconda vs Python officiel*

| Critère | Anaconda/Miniconda | Python officiel |
|---|---|---|
| Gestion d'environnements | `conda` (intégré) | `venv` (standard) |
| Installation | plus lourde (Anaconda), légère (Miniconda) | légère |
| Dépendances compilées (Data/ML) | souvent plus simple | parfois plus délicat |
| Usage IT typique | data/IA, notebooks, calcul | scripts, web, automation, CLI |
| Reproductibilité | `environment.yml` | `requirements.txt` |

**Étapes d'installation (quel que soit le choix)**

1. Télécharger l'installeur (Anaconda/Miniconda ou Python officiel).
2. Lancer l'installation et suivre l'assistant.
3. **Étape critique (Windows) :** ajouter Python au **PATH** (ou enregistrer Python par défaut).
4. Vérifier dans un terminal : `python -version`.

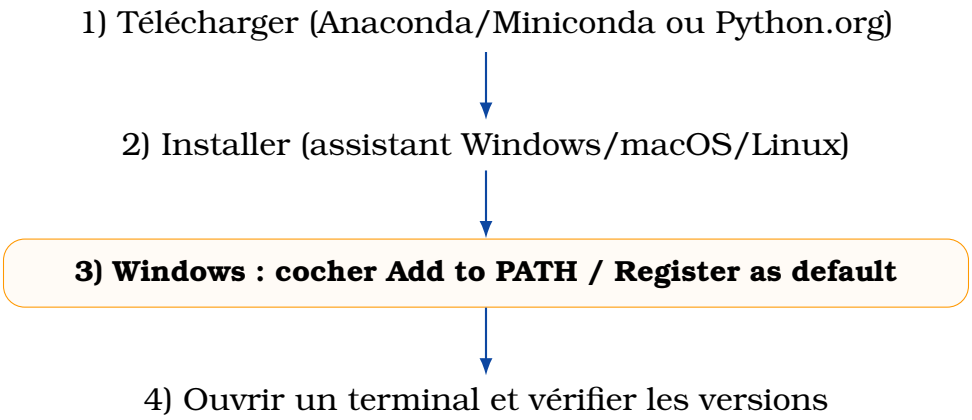



FIGURE 1.5 – *Workflow recommandé : installation + étape critique PATH + vérification.*

### Vérification : confirmer que l'environnement est prêt

Après l'installation, vérifiez que les commandes sont accessibles depuis un terminal.

Terminal

```
python --version
python -c "import sys; print(sys.executable)"
pip --version
# Si Anaconda/Miniconda :
conda --version
```

**Interprétation rapide**

- Si `python` ou `pip` est introuvable : problème de **PATH** ou d'environnement non activé.
- `sys.executable` indique le **Python réellement utilisé** (diagnostic n°1 en cas de confusion IDE/terminal).

### 1.2.2 Environnements virtuels et packages : robustesse et reproductibilité

**Environnement virtuel**

Un **environnement virtuel** est un espace isolé contenant un Python et des bibliothèques propres à un projet. L'activation redirige `python` et `pip` vers cet espace, évitant de « polluer » l'installation globale.

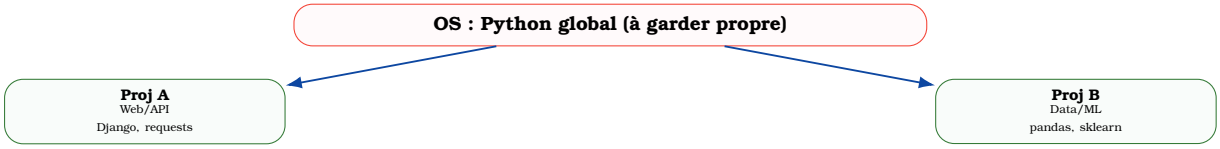



FIGURE 1.6 – *Un projet = un environnement.*

**Bénéfices IT**

- **Isolation** : pas de conflits de versions.
- **Reproductibilité** : recréation sur une autre machine.
- **Déploiement** : base propre CI/CD.

### `pip` vs `conda` : choisir comme un ingénieur IT

TABLE 1.2 – *`pip` et `conda` : rôle et limites*

| Outil | Points forts | Limites / vigilance |
|---|---|---|
| `pip` | Standard Python, énorme catalogue (PyPI) | gère surtout le Python ; dépendances natives parfois difficiles |
| `conda` | gère aussi binaires/dépendances natives ; environnements intégrés | plus « écosystème » conda ; mélange pip/conda à maîtriser |

//

Terminal

```
python -m pip install requests
python -m pip install --upgrade requests
python -m pip uninstall requests
python -m pip list
python -m pip freeze > requirements.txt
python -m pip install -r requirements.txt
```

//

Terminal

```
conda create -n dev_web python=3.11
conda activate dev_web
conda install flask numpy
conda env export > environment.yml
conda env create -f environment.yml
```

**Débogage : `ModuleNotFoundError`**

Cette erreur signifie : « le module n'est pas installé dans **l'environnement** qui exécute votre script ».

**Méthode en 3 étapes :**

1. Identifier le Python courant : `python -c "import sys; print(sys.executable)"`.
2. Installer dans ce Python : `python -m pip install nom_du_module`.
3. Relancer (et redémarrer l'IDE si nécessaire).

## 1.3 Découverte du REPL et premières commandes : Votre premier laboratoire Python

Le REPL (Read-Eval-Print Loop), ou interpréteur interactif Python, est votre premier outil pour dialoguer directement avec Python. Imaginez-le comme une calculatrice super-puissante où vous pouvez taper des instructions Python et obtenir un résultat immédiat. C'est l'endroit parfait pour expérimenter et tester vos premières lignes de code.

### 1.3.1 La console interactive Python (REPL) : Testez vos idées instantanément

Le REPL fonctionne sur un cycle simple et rapide : il **lit** ce que vous tapez, l'**évalue** (l'exécute), **affiche** le résultat, puis **boucle** pour la prochaine instruction.

Pour lancer le REPL, ouvrez un terminal (ou l'invite de commande sur Windows) et tapez `python` (ou `python3` si `python` est une version plus ancienne). Vous verrez apparaître le prompt `>>>`.

Terminal

```
python
>>> print("Bonjour, Python !") # Un message simple
Bonjour, Python !

>>> 5 + 3 # Un calcul arithmétique de base
8

>>> # Un calcul informatique simple : combien de
    kilooctets dans un mégaoctet ?
>>> 1024 * 1024
1048576

>>> "Python est " + "facile !" # On peut "coller" des
    textes
'Python est facile !'

>>> exit() # Pour quitter le REPL (ou Ctrl+D sur
    Linux/macOS, Ctrl+Z puis Entrée sur Windows)
```

**Pourquoi le REPL est utile pour commencer ?**

Le REPL est parfait pour :

- **Apprendre vite :** Tester des commandes Python sans avoir à créer un fichier.
- **Expérimenter :** Voir immédiatement le résultat de vos instructions.
- **Découvrir :** Explorer de nouvelles fonctions ou opérateurs.

### 1.3.2 Fonctions d'interaction de base : `print()` et `input()`

Pour qu'un programme soit utile, il doit pouvoir communiquer. Les fonctions `print()` et `input()` sont vos premières commandes pour que votre programme "parle" à l'utilisateur et "écoute" ce qu'il a à dire.

#### `print()` : Votre programme s'exprime

La fonction `print()` est la façon dont votre programme affiche du texte, des nombres, ou des résultats sur la console. C'est essentiel pour donner des informations à l'utilisateur ou pour vérifier ce que fait votre code.

In [0]:

```
## 1. Afficher un message de bienvenue
print("Bienvenue au cours de Python !")

# 2. Afficher le résultat d'un calcul
nombre_pommes = 10
nombre_oranges = 5
print("J'ai", nombre_pommes + nombre_oranges, "fruits.")

# 3. Utiliser les f-strings pour un message clair (méthode moderne)
# Les f-strings (f"...") sont pratiques pour insérer des variables directement dans le texte.
nom_utilisateur = "Alice"
print(f"Bonjour, {nom_utilisateur} ! Ravi de te voir.")
```

Out[0]:

```
Bienvenue au cours de Python ! J'ai 15 fruits. Bonjour, Alice ! Ravi de te voir.
```

**L'astuce des f-strings**

Les **f-strings** (chaînes de caractères formatées, préfixées par `f` comme `f"Mon texte ma_variable"`) sont la méthode la plus simple et moderne pour afficher des messages combinant du texte et des variables. Adoptez-les dès maintenant !

### `input()` : Votre programme écoute l'utilisateur

La fonction `input()` permet à votre programme de demander une information à l'utilisateur et d'attendre sa réponse. La réponse est ensuite stockée dans une variable.

In [0]:

```
## 1. Demander le nom de l'utilisateur
    prenom = input("Quel est votre prénom ? ")
    print(f"Bonjour, {prenom} !")

    # 2. Demander une information numérique (attention au type !)
    annee_naissance_str = input("En quelle année êtes-vous né(e) ? ")
    # La fonction input() renvoie TOUJOURS du texte (une "chaîne de caractères").
    # Pour faire des calculs, il faut convertir ce texte en nombre.
    annee_naissance_int = int(annee_naissance_str) # Convertit le texte en nombre entier
    age_approx = 2024 - annee_naissance_int # Exemple de calcul simple
    print(f"Vous avez environ {age_approx} ans.")
```

Out[0]:

```
Quel est votre prénom ? Léa Bonjour, Léa ! En quelle année êtes-vous né(e) ? 2003 Vous avez environ 21 ans.
```

**Attention au type de données avec `input()`**

`input()` renvoie toujours une **chaîne de caractères** (du texte). Si vous attendez un nombre pour faire des calculs, vous devez le **convertir explicitement** en utilisant `int()` (pour les nombres entiers) ou `float()`

(pour les nombres à virgule). Si l'utilisateur tape du texte non numérique pour une conversion, votre programme produira une erreur (`ValueError`). Nous verrons comment gérer ces erreurs plus tard.

## 1.4 Environnements de développement (IDEs) : Votre poste de travail Python complet

Bien que le REPL soit excellent pour l'expérimentation rapide, l'écriture et la gestion de projets Python plus complexes (applications web, scripts d'automatisation avancés, systèmes d'analyse de données) nécessitent un outil plus sophistiqué : un **IDE** (Integrated Development Environment - Environnement de Développement Intégré).

Un IDE regroupe en une seule interface des outils essentiels pour toutes les phases du développement logiciel, augmentant considérablement votre productivité et la qualité de votre code.

### 1.4.1 Fonctionnalités clés d'un IDE et leur utilité en IT

Un bon IDE Python offre un ensemble de fonctionnalités qui transforment votre expérience de codage :

- **Éditeur de code intelligent :**
  - **Coloration syntaxique :** Met en évidence les mots-clés, chaînes, commentaires. Facilite la lecture du code.
  - **Autocomplétion (IntelliSense) :** Suggère les noms de variables, fonctions, classes et méthodes. Réduit les erreurs de frappe et accélère l'écriture du code.
  - **Vérification d'erreurs en temps réel (Linting) :** Souligne les erreurs de syntaxe ou les avertissements de style pendant que vous écrivez. Permet de corriger les problèmes avant même d'exécuter le code.
  - **Refactoring :** Outils pour renommer des variables/fonctions à travers tout le projet, extraire du code en fonctions, etc. Maintient la qualité du code et la structure du projet.

  **Utilité IT :** Accélère le développement, réduit les bugs, et améliore la maintenabilité du code, des points cruciaux pour la livraison rapide et la stabilité des applications.

- **Débogueur intégré : Utilité IT :** Permet d'exécuter le code pas à pas, de définir des points d'arrêt (*breakpoints*) pour inspecter l'état des variables à un moment précis, et de détecter les erreurs logiques. Indispensable pour trouver et corriger des bugs complexes dans des systèmes IT.
- **Gestion de projet et de fichiers : Utilité IT :** Organise vos fichiers, dossiers, dépendances et environnements virtuels. Simplifie la navigation dans de grands projets et l'intégration avec les systèmes de contrôle de version (Git).
- **Console interactive intégrée : Utilité IT :** Combinaison des avantages du REPL avec la gestion du projet. Permet de tester des extraits de code dans le contexte de votre application.
- **Intégration de Terminal : Utilité IT :** Accès direct à la ligne de commande pour exécuter des scripts Python, gérer les paquets avec `pip`/`conda` ou utiliser Git, sans quitter l'IDE.

### 1.4.2 Les IDEs Python populaires pour les étudiants en IT

Plusieurs IDEs sont populaires et adaptés au développement Python. Votre choix dépendra de vos préférences, du type de projet et des ressources de votre machine.

**PyCharm : L'IDE de référence pour le développement Python professionnel**

PyCharm

Développé par JetBrains, PyCharm est un IDE puissant et très apprécié, particulièrement adapté aux projets Python de grande envergure et aux équipes professionnelles. Il existe en version gratuite (Community) et payante (Professional, avec des fonctionnalités avancées pour le web, la data science, etc.).

**Points forts pour l'IT :**

- **Analyse de code intelligente :** Détecte les erreurs, les problèmes de style (PEP 8) et les vulnérabilités potentielles.
- **Débogueur avancé :** Un des meilleurs débogueurs du marché pour Python, essentiel pour les applications complexes.
- **Intégration poussée :** Support natif pour les frameworks web (Django, Flask), les bases de données, Docker, Git.
- **Environnements virtuels :** Gestion fluide de `venv` et `conda`.

**Installation :** Téléchargez la version Community sur https://www.jetbrains.com/pycharm/download/. L'assistant d'installation est intuitif et vous guidera pour configurer votre interpréteur Python.

### VS Code (Visual Studio Code) : La flexibilité au service du développeur Python

VS Code

Visual Studio Code de Microsoft est un éditeur de code open source, léger et multiplateforme. Sa force réside dans son écosystème d'extensions très riche qui lui permet de se transformer en un IDE complet pour Python (et bien d'autres langages). Il est extrêmement populaire pour sa rapidité, sa flexibilité et sa grande personnalisation.

**Points forts pour l'IT :**

- **Léger et rapide :** Démarrage instantané, faible consommation de ressources.
- **Extensions puissantes :** L'extension officielle "Python" de Microsoft (avec Pylance pour l'autocomplétion) offre une expérience IDE complète.
- **Polyvalence :** Idéal pour les scripts d'automatisation, le développement web (front-end/back-end), la gestion de configuration (DevOps), et l'analyse de logs.
- **Intégration Git :** Support natif et intuitif pour le contrôle de version.

**Installation :**

1. Téléchargez VS Code sur https://code.visualstudio.com/.
2. Installez l'extension officielle "Python" de Microsoft via le gestionnaire d'extensions intégré (accessible via Ctrl+Shift+X).

### Jupyter Notebook / JupyterLab : L'environnement interactif pour la Data Science et l'analyse

Jupyter Notebook est un environnement de développement web interactif qui permet de créer des documents appelés "notebooks". Ces notebooks sont des documents "vivants" qui mélangent :

- Du code exécutable (Python, R, Julia).

- Du texte formaté (Markdown), des équations (LaTeX).
- Des visualisations (graphiques, tableaux) et des médias.

JupyterLab est une version plus avancée et modulable de Jupyter Notebook, offrant une interface plus complète avec des onglets, un explorateur de fichiers et la possibilité d'ouvrir différents types de documents.

**Points forts pour l'IT (Data Science  Recherche) :**

- **Interactivité par cellule :** Exécutez le code cellule par cellule, parfait pour l'exploration de données, l'expérimentation et le prototypage rapide d'algorithmes.
- **Affichage Riche :** Affiche directement les graphiques (Matplotlib, Seaborn), les tableaux (Pandas) et les résultats de calculs inline, idéal pour l'analyse exploratoire des données.
- **Documentation Intégrée et Reproductible :** Combinez du code avec des explications textuelles en Markdown, créant des rapports techniques interactifs et reproductibles.
- **Partage Facile :** Les fichiers `.ipynb` sont faciles à partager et peuvent être ouverts et exécutés par d'autres utilisateurs via Jupyter ou des plateformes comme Google Colab.

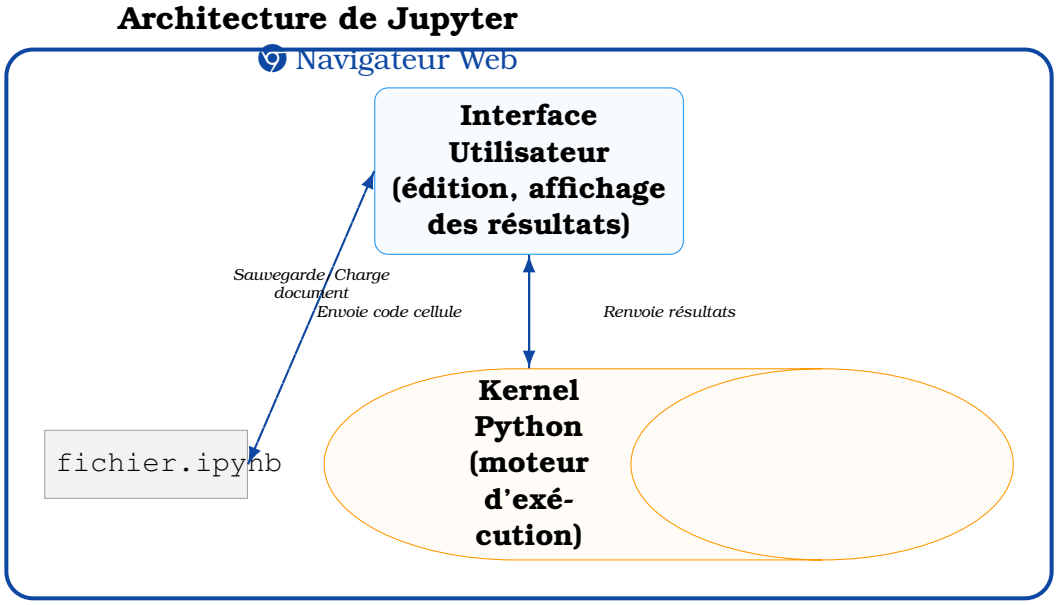



**FIGURE 1.7 –** *Fonctionnement interactif de Jupyter Notebook/Lab, idéal pour l'analyse de données et la recherche IT.*

**Installation et Lancement :**

- **Si Anaconda est installé :** Jupyter Notebook et JupyterLab sont généralement déjà inclus. Lancez-les depuis le "Anaconda Navigator" ou via votre terminal :

```
Terminal
jupyter notebook
# ou pour JupyterLab
jupyter lab
```

**Si Python officiel est installé (avec pip) :**

```
Terminal
pip install notebook
# ou pour l'interface plus riche de JupyterLab
pip install jupyterlab
```

Puis lancez avec `jupyter notebook` ou `jupyter lab`.

TABLE 1.3 – *Comparatif des environnements de développement Python pour les professionnels de l'IT*

| Environnemen | Caractéristiques Principales | Scénarios d'Utilisation Idéaux en IT |
|---|---|---|
| **REPL (Console)** | Exécution immédiate, ligne par ligne, retour instantané. | Tests rapides de syntaxe/-fonctions, exploration d'APIs, debugging de snippets. |
| **Jupyter Notebook / JupyterLab** | Interactif, mélange code/texte/graphiques, exécution par cellule. | Analyse exploratoire de données, rapports d'audit interactifs, prototypage ML/IA, tutoriels. |
| **PyCharm** | IDE complet : débogueur avancé, refactoring, gestion de projet, intégration Git. | Développement d'applications web (Django/Flask), microservices, backend complexes, grands projets d'équipe. |
| **VS Code** | Éditeur léger et extensible : extensions riches, support multi-langages, terminal intégré. | Scripts d'automatisation, développement web (front/-back), gestion de configuration (DevOps), analyse de logs. |
| **Spyder** | Orienté scientifique : explorateur de variables, console IPython, éditeur Matplotlib. | Data Science traditionnelle, analyse statistique, simulations (ingénierie, finance), bioinformatique. |
| **Google Colab** | Notebooks basés sur le cloud : GPU/TPU gratuits, partage facile, pas d'installation locale. | Projets Deep Learning collaboratifs, prototypage rapide d'IA, apprentissage à distance sans infrastructure matérielle. |

## 1.5 Structure d'un programme Python : Organiser votre premier code

Écrire du code ne se résume pas à taper des instructions ; il s'agit aussi de les organiser de manière logique pour qu'elles soient comprises par la machine ET par les humains. En IT, un programme bien structuré est la clé de la **maintenabilité** et de la **collaboration**.

### 1.5.1 L'Indentation : La règle d'or de Python

En Python, l'**indentation** n'est pas juste pour la beauté du code, c'est une **règle de syntaxe** ! Elle définit les blocs de code. Imaginez-la comme les paragraphes d'un texte : chaque nouveau paragraphe (bloc de code) commence avec une indentation.

**IndentationError : L'erreur fréquente du débutant**

Une `IndentationError` survient si votre code n'est pas indenté correctement (par exemple, un mélange d'espaces et de tabulations, ou une indentation incohérente). L'usage recommandé est de **4 espaces** par niveau d'indentation. Votre IDE configurera cela automatiquement.

In [0]:

```
## Un code simple pour illustrer l'indentation
    serveur_actif = True

    if serveur_actif: # Si le serveur est actif...
    print("Le serveur est opérationnel.") # ... alors exécute cette ligne (indentée)
    print("Vérification en cours...")     # ... et celle-ci (même indentation)
    print("Fin de la vérification.")      # Cette ligne s'exécute toujours, elle n'est pas indentée
```

Out[0]:

```
Le serveur est opérationnel. Vérification en cours... Fin de la vérification.
```

**Utilité IT de l'indentation :** L'indentation obligatoire rend le code Python naturellement lisible, réduisant les erreurs de logique et facilitant la compréhension mutuelle au sein d'une équipe IT.

### 1.5.2 Les commentaires : Votre code parle aux humains

Les commentaires sont des notes dans votre code destinées aux humains (vous, et vos collègues !). L'interpréteur Python les ignore. Ils sont essentiels pour expliquer ce que fait une partie complexe du code, pourquoi un choix a été fait, ou pour laisser une note importante.

In [0]:

```
## Ceci est un commentaire sur une seule ligne.
    # Il explique la ligne de code suivante.
```

```
3
4       # Déclaration d'une variable pour stocker le nom de
            l'application
5       nom_application = "GestionRH"
6
7       # Affichage d'un message de démarrage de l'application
8       print(f"Démarrage de l'application :
            {nom_application}")
9
10      # Les commentaires peuvent aussi être placés après une
            instruction
11      version = "1.0.0" # Numéro de version actuel
12      print(f"Version : {version}")
```

Out[0]:

```
Démarrage de l'application : GestionRH Version : 1.0.0
```

**L'art du commentaire efficace en IT**

- **Expliquez le "pourquoi", pas le "quoi" :** Le code explique souvent le "quoi". Le commentaire doit expliquer le "pourquoi" (ex : "Pourquoi utiliser cette technique plutôt qu'une autre ?").
- **Soyez concis et pertinent :** Des commentaires trop longs ou évidents sont contre-productifs.

**Utilité IT :** Les commentaires sont cruciaux pour la **maintenabilité** du code et le **partage des connaissances** dans les équipes IT. Ils aident les nouveaux membres à comprendre rapidement le projet.

### 1.5.3 Point d'entrée d'un script : Votre programme commence ici

Pour des scripts simples écrits dans un seul fichier, Python exécute le code de haut en bas. Cependant, une bonne pratique existe pour définir explicitement le "point de départ" de votre script. Ce n'est pas une obligation pour les scripts très simples, mais c'est une convention professionnelle que vous verrez souvent.

```
mon_premier_script.py

    # 1. Zone de définition (fonctions, variables globales, etc.)
    message_bienvenue = "Bienvenue dans votre premier script Python !"

    def afficher_infos_systeme():
    # Cette fonction sera implémentée plus tard pour afficher des infos système
    return "Informations système fictives."

    # 2. Point d'entrée principal du script
    # Ce bloc s'exécute UNIQUEMENT si le script est lancé directement
    if __name__ == "__main__":
    print(message_bienvenue)
    infos = afficher_infos_systeme()
    print(infos)
    print("Script terminé.")
```

**Utilité IT de ce bloc (`if __name__ == "__main__":`) :**

- **Clarté structurelle :** Il indique clairement où commence l'exécution principale de votre script.
- **Flexibilité future :** Sans rentrer dans les détails techniques complexes de "modules" pour le moment, sachez que ce bloc permet à votre script d'être potentiellement réutilisé par d'autres programmes sans exécuter sa logique principale.

## 1.6 Variables et types de données de base : Les fondations de l'information IT

En programmation, manipuler l'information est au cœur de tout ce que vous ferez en IT. Pour cela, vous aurez besoin de deux concepts fondamentaux : les **variables** pour stocker cette information, et les **types de données** pour définir sa nature. Comprendre ces concepts est la première étape pour écrire des programmes qui gèrent des données de manière efficace et sans erreur.

### 1.6.1 Variables : Conteneurs pour vos données

Imaginez une variable comme une **boîte nommée** dans la mémoire de votre ordinateur. Vous pouvez y ranger une valeur, et cette valeur peut changer au cours de l'exécution de votre programme.

**Variable en programmation**

Une **variable** est un emplacement nommé dans la mémoire de l'ordinateur qui permet de stocker une valeur. Son contenu peut être modifié pendant l'exécution d'un programme.

#### Déclaration et affectation en Python

En Python, la déclaration (créer la boîte) et l'affectation (y mettre une valeur) se font en une seule étape simple, en utilisant l'opérateur `=`. Python est un langage à **typage dynamique** : vous n'avez pas besoin de spécifier le type de la variable (texte, nombre, etc.) à l'avance ; Python le déduit de la valeur que vous lui donnez.

In [0]:

```
## Affectation de différentes valeurs à des variables
    nombre_utilisateurs_actifs = 150 # Un nombre entier
    nom_application = "Gestion de Stock" # Une chaîne de
        caractères
    est_service_demarre = True # Une valeur booléenne
        (vrai/faux)

    # Affichage des valeurs des variables
    print("Utilisateurs actifs:",
        nombre_utilisateurs_actifs)
    print("Application courante:", nom_application)
    print("Service démarré:", est_service_demarre)

    # La valeur d'une variable peut être modifiée à tout
        moment
    nombre_utilisateurs_actifs = 155
    print("Nouvel affichage des utilisateurs:",
        nombre_utilisateurs_actifs)
```

Out[0]:

```
Utilisateurs actifs: 150 Application courante: Gestion de Stock
Service démarré: True Nouvel affichage des utilisateurs: 155
```

**Règles et bonnes pratiques pour les noms de variables en IT**

Choisir des noms de variables **clairs et descriptifs** est crucial pour la lisibilité et la maintenabilité de votre code, surtout quand vous travaillez en équipe sur des projets IT :

- **Descriptifs :** `nom_utilisateur` est bien plus explicite que `n` ou `x`.
- **Minuscules avec underscores (`snake_case`) :** C'est la convention Python (ex : `nombre_elements`, `chemin_fichier_log`).
- **Pas de caractères spéciaux ou d'espaces :** Utilisez _ à la place des espaces.
- **Ne pas commencer par un chiffre :** `1_variable` est invalide, mais `variable_1` est valide.
- **Éviter les mots-clés réservés de Python :** (ex : `print`, `if`, `for`, `class`).

### 1.6.2 Types de données de base : La nature de l'information

Le **type de données** d'une variable indique la catégorie de valeur qu'elle contient (nombre, texte, vrai/faux, etc.). Bien que Python détermine les types automatiquement, comprendre leur fonctionnement est essentiel pour :

- Choisir le bon type pour stocker vos données efficacement.
- Effectuer les opérations correctes sur ces données.
- Éviter les erreurs de type dans vos programmes.

Vous pouvez connaître le type de n'importe quelle variable ou valeur en utilisant la fonction `type()`.

#### Nombres : `int` (entiers) et `float` (flottants)

Les nombres sont le pain quotidien de tout programme IT, que ce soit pour compter des ressources, calculer des performances ou manipuler des données financières.

- **Entiers (`int`) :** Représentent des nombres entiers, positifs ou négatifs, sans partie décimale. **Usages IT :** Compteurs (nombre de requêtes), identifiants (ID utilisateur), ports réseau (80, 443), adresses IP (parties).
- **Flottants (`float`) :** Représentent des nombres réels, avec une partie décimale. **Usages IT :** Mesures précises (température CPU : 65.5°C), calculs de latence réseau (12.345 ms), taux et pourcentages (taux d'erreur : 0.0125).

In [0]:

```
## Exemples de nombres entiers et flottants
    nombre_machines_virtuelles = 50
    version_os = 10
    print(f"Type de 'nombre_machines_virtuelles':
        {type(nombre_machines_virtuelles)}")
    print(f"Type de 'version_os': {type(version_os)}")

    latence_moyenne_ms = 12.34
    taux_utilisation_cpu = 0.855 # 85.5%
    print(f"Type de 'latence_moyenne_ms':
        {type(latence_moyenne_ms)}")
    print(f"Type de 'taux_utilisation_cpu':
        {type(taux_utilisation_cpu)}")
```

Out[0]:

$$\texttt{Type de 'nombre}_{m}achines_{v}irtuelles' :< class'int' > Typede'version_{o}s' :< class'int' > Typede'latence_{m}oyenne_{m}s' :< class'float' > Typede'taux_{u}tilisation_{c}pu' :< class'float' >$$

**Attention à la précision des flottants en IT**

Les nombres flottants peuvent parfois avoir de très petites imprécisions dues à la manière dont les ordinateurs stockent les nombres à virgule. Pour la plupart des usages, c'est transparent, mais pour des calculs financiers critiques ou des comparaisons d'égalité strictes, soyez vigilant. Pour une précision décimale exacte, Python offre le module `decimal`.

### Chaînes de caractères (`str`) : Manipuler le texte et les informations structurées

Le texte est omniprésent en IT. Les chaînes de caractères (`str`) sont utilisées pour stocker et manipuler toutes sortes d'informations textuelles.

- **Déclaration :** Les chaînes sont délimitées par des guillemets simples (`'`) ou doubles (`""`).
- **Usages IT :** Noms de fichiers, messages de logs, adresses IP, URLs, noms d'utilisateurs, mots de passe (avant hachage !), requêtes SQL.

In [0]:

```
## Déclaration de chaînes de caractères
```

```
    nom_fichier_log = "access_2024-03-15.log"
    message_alerte = 'Alerte sécurité: Tentative de
        connexion suspecte.'
    adresse_ip_source = "192.168.1.10"
    print(f"Type de 'nom_fichier_log':
        {type(nom_fichier_log)}")
    print(f"Adresse IP source: {adresse_ip_source}")

    # Concaténation de chaînes (assembler plusieurs
        chaînes)
    chemin_complet_log = "/var/log/apache/" +
        nom_fichier_log
    print(f"Chemin complet du log: {chemin_complet_log}")

    # Longueur d'une chaîne avec len()
    print(f"La longueur du message d'alerte est de
        {len(message_alerte)} caractères.")
```

Out[0]:

```
Type de 'nom_fichier_log': <class 'str'> Adresse
IP source: 192.168.1.10 Chemin complet du log:
/var/log/apache/access_2024-03-15.log La longueur du message
d'alerte est de 53 caractères.
```

**Chaînes et sécurité IT**

La manipulation de chaînes provenant d'utilisateurs est une source majeure de vulnérabilités. **Validez et nettoyez toujours les entrées utilisateur** (ex : pour éviter les injections SQL ou les attaques XSS). Ne stockez jamais de mots de passe en clair.

### Booléens (`bool`) : La logique Vrai/Faux

Les booléens sont le fondement de la prise de décision en informatique. Une variable booléenne ne peut avoir que deux valeurs : `True` (vrai) ou `False` (faux).

- **Usages IT :** Représenter l'état d'un système (serveur actif ?), le résultat d'une condition (utilisateur authentifié ?), l'activation d'une fonctionnalité (mode maintenance activé ?).

In [0]:

```
## Exemples de variables booléennes
    serveur_principal_actif = True
    sauvegarde_terminee_avec_erreurs = False
    print(f"Type de 'serveur_principal_actif':
        {type(serveur_principal_actif)}")
    print(f"Sauvegarde réussie sans erreur: {not
        sauvegarde_terminee_avec_erreurs}") # Utilisation
        de 'not'

    # Les comparaisons renvoient des booléens
    charge_cpu = 95
    seuil_critique = 90
    alerte_cpu = charge_cpu > seuil_critique # La
        condition est-elle vraie ?
    print(f"Alerte CPU déclenchée: {alerte_cpu}")
```

Out[0]:

```
Type de 'serveur_principal_actif': <class 'bool'> Sauvegarde
réussie sans erreur: True Alerte CPU déclenchée: True
```

**Logique binaire et informatique**

L'informatique est basée sur la logique binaire (0 ou 1, True ou False). Comprendre les booléens est la première étape pour écrire des programmes capables de prendre des décisions complexes basées sur l'état de votre système IT.

### 1.6.3 Conversion de types (Type Casting) : Adapter les données

En IT, les données que vous recevez ne sont pas toujours dans le format idéal. La **conversion de type** (ou *type casting*) est l'opération qui consiste à transformer une valeur d'un type de données vers un autre. C'est indispensable pour manipuler des données provenant de saisies utilisateur, de fichiers ou de bases de données.

In [0]:

```
## 1. Conversion d'une chaîne de caractères en nombre (le
    plus fréquent)
```

```
    port_config_str = "443" # Port reçu d'un fichier de configuration (c'est du texte)
    port_config_int = int(port_config_str) # On le convertit en entier pour pouvoir faire des calculs ou des comparaisons
    print(f"Port en entier : {port_config_int}, type : {type(port_config_int)}")

    charge_cpu_str = "85.7" # Charge CPU lue d'un capteur (texte)
    charge_cpu_float = float(charge_cpu_str) # On le convertit en flottant
    print(f"Charge CPU en flottant : {charge_cpu_float}, type : {type(charge_cpu_float)}")

    # 2. Conversion d'un nombre en chaîne (pour affichage ou concaténation)
    id_transaction = 987654
    log_message = "Transaction ID: " + str(id_transaction) + " effectuée avec succès."
    print(log_message)

    # 3. Conversion en booléen
    # int en bool : 0 devient False, tout autre nombre devient True
    # str en bool : chaîne vide ("") devient False, toute autre chaîne devient True
    bool_zero = bool(0)
    bool_non_zero = bool(50)
    bool_empty_str = bool("")
    bool_non_empty_str = bool("OK")
    print(f"bool(0): {bool_zero}, bool(50): {bool_non_zero}, bool(''): {bool_empty_str}, bool('OK'): {bool_non_empty_str}")
```

**Out[0]:**

```
Port en entier : 443, type : <class 'int'> Charge CPU en flottant : 85.7, type : <class 'float'> Transaction ID: 987654 effectuée avec succès. bool(0): False, bool(50): True, bool(''): False, bool('OK'): True
```

**⚠ Risque de `ValueError` lors de la conversion : L'importance du contrôle d'entrée**

Une conversion de type échoue si la valeur ne peut pas être transformée dans le type cible. Par exemple, `int("bonjour")` provoquera une `ValueError` car "bonjour" n'est pas un nombre entier valide. En IT, c'est un point critique, surtout lors de la lecture de saisies utilisateur ou de données provenant de sources externes. La gestion de ces erreurs (via les blocs `try-except`) est essentielle pour rendre vos programmes robustes et tolérants aux pannes, et sera abordée dans un chapitre ultérieur.

## 1.7 Travaux Pratiques (TP) du Module 1 : Mettre les mains dans le code

Ces exercices pratiques sont conçus pour vous permettre d'appliquer les connaissances acquises dans ce module. Ils couvrent l'installation, l'utilisation du REPL, les premières commandes, la gestion des packages et la structuration de base d'un programme Python.

[TP 1 : Vérification d'environnement et première interaction] **Objectif IT :** S'assurer que votre poste de travail est prêt pour le développement Python et réaliser vos premières opérations interactives.

1. Ouvrez un terminal (ou l'invite de commande sur Windows).
2. Vérifiez la version de Python installée avec la commande appropriée.
3. Lancez l'interpréteur Python (REPL).
4. Dans le REPL, effectuez les opérations suivantes :
   - Affichez un message de bienvenue de votre choix.
   - Calculez la taille en octets d'un fichier de 2.5 Go (1 Go = 1024 Mo, 1 Mo = 1024 Ko, 1 Ko = 1024 octets).
   - Concaténez votre prénom et votre nom pour former une adresse email fictive (ex : "prenom.nom@univ.com").
   - Quittez le REPL.
5. Prenez une capture d'écran de votre terminal montrant toutes ces étapes et les résultats.

[TP 2 : Création de votre premier script d'information système] **Objectif IT :** Écrire, exécuter et structurer un script Python simple pour collecter des informations.

1. Ouvrez votre IDE préféré (VS Code ou PyCharm).
2. Créez un nouveau fichier nommé `infos_systeme.py`. Dans ce fichier, ajoutez des commentaires
3. Déclarez une variable `nom_utilisateur` et affectez-lui une valeur saisie par l'utilisateur.
4. Déclarez une variable `version_os` et affectez-lui une valeur saisie par l'utilisateur (ex : "Windows 11" ou "Ubuntu 22.04").
5. Dans le bloc `if __name__ == "__main__":` :
   - Affichez un message de bienvenue à l'utilisateur, incluant son nom.
   - Affichez les informations sur le système d'exploitation qu'il a saisies.
   - Ajoutez un message de fin de script.
6. Exécutez le script depuis votre IDE ou le terminal.
7. Copiez le code de votre script et la sortie de son exécution dans un document texte.

[TP 3 : Gestion d'un environnement virtuel et installation de packages] **Objectif IT :** Maîtriser la création d'environnements virtuels et l'installation de bibliothèques, compétence essentielle en gestion de dépendances.

1. Ouvrez votre terminal.
2. Créez un nouvel environnement virtuel nommé `mon_projet_tp`.
3. Si vous utilisez Anaconda/Miniconda : `conda create -n mon_projet_tp python=3.11` (ou la dernière version stable).
4. Si vous utilisez Python officiel : `python -m venv mon_projet_tp`.

Activez cet environnement virtuel.

Vérifiez que Python et pip pointent bien vers cet environnement (utilisez `python -c "import`

Installez la bibliothèque `requests` (utilisée pour les requêtes HTTP en Python) dans cet environnement :

- Si `conda` : `conda install requests`.
- Si `pip` : `pip install requests`.

Lister les packages installés dans cet environnement.

Désactivez l'environnement.

Prenez une capture d'écran de toutes les commandes et leurs sorties dans votre terminal.

[TP 4 : Utilisation de Jupyter Notebook/Lab pour une petite analyse] **Objectif IT :** Se familiariser avec l'environnement interactif Jupyter pour l'exploration de données ou la documentation.

1. Lancez Jupyter Notebook ou JupyterLab.
2. Créez un nouveau Notebook Python (fichier `.ipynb`).
3. Dans la première cellule (type Markdown) :
   - Écrivez un titre de niveau 1 : " Mon Premier Notebook IT".
   - Ajoutez un paragraphe expliquant brièvement le but du notebook (ex : "Ce notebook explore les capacités de Python pour des calculs IT simples.").
4. Dans la deuxième cellule (type Code) :
   - Déclarez une variable `debit_mbps` et affectez-lui la valeur 100 (pour 100 Mbps).
   - Déclarez une variable `taille_fichier_mo` et affectez-lui la valeur 500 (pour 500 Mo).
   - Calculez et affichez le temps de transfert en secondes (1 Mo = 8 Mbits).
   - Utilisez la fonction `print()` avec une f-string pour un affichage clair.
5. Exécutez toutes les cellules du notebook.
6. Sauvegardez le notebook et prenez une capture d'écran du résultat final dans votre navigateur.

[TP 5 : Débogage simple et corrections d'erreurs d'indentation/syntaxe] **Objectif IT :** Apprendre à identifier et corriger les erreurs de base ('IndentationError', 'NameError', 'TypeError'), compétences essentielles en débogage.

1. Copiez le code Python suivant dans un nouveau fichier `bug_catcher.py` :

Exécutez le script. Vous devriez obtenir une erreur.

**Identifiez la première erreur dans le traceback** (le message d'erreur). De quel type d'erreur s'agit-il ?

Corrigez cette première erreur (*IndentationError*).

Réexécutez le script. Identifiez la nouvelle erreur. De quel type d'erreur s'agit-il ?

Corrigez toutes les erreurs une par une jusqu'à ce que le script s'exécute sans erreur et affiche un résultat cohérent. (Indice : il y a une erreur de type pour l'addition).

Copiez le code final corrigé et la sortie d'exécution dans un document, en expliquant pour chaque erreur son type et la correction apportée.

## 1.8 Quiz du Module 1 : Testez vos connaissances sur Python et son environnement

Ce quiz vous permettra de vérifier votre compréhension des concepts fondamentaux abordés dans ce module.

**Q.1.** **Vrai/Faux :** Python est un langage compilé, ce qui signifie que le code est transformé en un exécutable binaire avant toute exécution.

- ☑ Vrai
- ☑ Faux

**Q.2.** **QCM :** Parmi les propositions suivantes, quelle est la raison principale de l'importance de l'indentation en Python pour un programmeur IT ?

Elle est juste une convention de style pour rendre le code plus joli.

Elle indique à l'interpréteur Python où commencent et finissent les blocs de code (conditions, boucles, etc.).

Elle permet d'accélérer l'exécution du code sur des processeurs multi-cœurs.

Elle est utilisée pour définir les types de variables.

**Q.3.** **Question Ouverte :** Expliquez brièvement l'expression "batteries included" dans le contexte de Python et donnez un exemple de son utilité pratique pour un développeur IT.

**Q.4.** **Vrai/Faux :** La fonction Python `input()` renvoie une valeur numérique par défaut si l'utilisateur saisit des chiffres.

- ☑ Vrai
- ☑ Faux

**Q.5.** **QCM :** Quel est l'objectif principal de l'utilisation d'environnements virtuels (`venv` ou `conda env`) dans un projet Python IT ?

Accélérer la vitesse d'exécution du code Python.

Isoler les dépendances spécifiques à un projet pour éviter les conflits de versions.

Rendre le code Python compatible avec des systèmes d'exploitation différents.

Installer les pilotes de cartes graphiques pour le Machine Learning.

**Q.6. Vrai/Faux :** Le REPL (Read-Eval-Print Loop) est principalement utilisé pour écrire des applications web complètes en production.

- ☑ Vrai
- ☑ Faux

**Q.7. Question Ouverte :** Citez au moins trois domaines d'application concrets où Python est très utilisé dans le secteur de l'Information Technology.

**Q.8. QCM :** Quelle commande est la meilleure pratique pour installer la bibliothèque `numpy` dans un environnement virtuel `conda` actif?

- `apt-get install numpy`
- `pip install numpy`
- `conda install numpy`
- `python setup.py install numpy`

**Q.9. Vrai/Faux :** Si Python n'est pas ajouté au PATH lors de l'installation sur Windows, vous ne pourrez jamais l'exécuter depuis la ligne de commande.

- ☑ Vrai
- ☑ Faux

**Q.10. Question Ouverte :** Vous rencontrez l'erreur `ModuleNotFoundError: No module named 'requests'`. Expliquez en 2 ou 3 étapes comment résoudre ce problème dans un environnement Python actif.

# Table des matières détaillée